

Fever Diagnosis and Medicine Dataset

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: df = pd.read_excel("Downloads/Fever Diagnosis and Medicine Dataset.xlsx")
```

```
In [5]: df
```

```
Out[5]:
```

	Temperature	Fever_Severity	Age	Gender	BMI	Headache	Body_Ache	Fatigue	Chr
0	36.1	Normal	89	Female	24.0	No	Yes	No	
1	37.5	Mild Fever	94	Male	26.6	No	No	No	
2	36.4	Normal	92	Male	27.8	No	No	No	
3	39.8	High Fever	66	Male	18.7	No	Yes	No	
4	39.3	High Fever	28	Male	21.0	No	Yes	No	
...	...	...	...	...	...	...	...	...	
995	39.9	High Fever	59	Male	22.1	Yes	Yes	Yes	
996	36.9	Normal	74	Female	18.1	Yes	Yes	Yes	
997	36.1	Normal	8	Female	29.6	Yes	Yes	No	
998	38.8	High Fever	87	Female	25.2	No	No	No	
999	38.2	High Fever	65	Female	26.0	No	Yes	No	

1000 rows × 20 columns



```
In [6]: df.shape
```

```
Out[6]: (1000, 20)
```

```
In [7]: df.head()
```

Out[7]:

	Temperature	Fever_Severity	Age	Gender	BMI	Headache	Body_Ache	Fatigue	Chron
0	36.1	Normal	89	Female	24.0	No	Yes	No	
1	37.5	Mild Fever	94	Male	26.6	No	No	No	
2	36.4	Normal	92	Male	27.8	No	No	No	
3	39.8	High Fever	66	Male	18.7	No	Yes	No	
4	39.3	High Fever	28	Male	21.0	No	Yes	No	

In [8]: `df.columns`

Out[8]: Index(['Temperature', 'Fever_Severity', 'Age', 'Gender', 'BMI', 'Headache', 'Body_Ache', 'Fatigue', 'Chronic_Conditions', 'Allergies', 'Smoking_History', 'Alcohol_Consumption', 'Humidity', 'AQI', 'Physical_Activity', 'Diet_Type', 'Heart_Rate', 'Blood_Pressure', 'Previous_Medication', 'Recommended_Medication'], dtype='object')

In [9]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Temperature                          1000 non-null   float64
1   Fever_Severity                       1000 non-null   object
2   Age                                  1000 non-null   int64
3   Gender                              1000 non-null   object
4   BMI                                  1000 non-null   float64
5   Headache                            1000 non-null   object
6   Body_Ache                           1000 non-null   object
7   Fatigue                             1000 non-null   object
8   Chronic_Conditions                   1000 non-null   object
9   Allergies                           1000 non-null   object
10  Smoking_History                      1000 non-null   object
11  Alcohol_Consumption                  1000 non-null   object
12  Humidity                             1000 non-null   float64
13  AQI                                  1000 non-null   int64
14  Physical_Activity                    1000 non-null   object
15  Diet_Type                            1000 non-null   object
16  Heart_Rate                           1000 non-null   int64
17  Blood_Pressure                       1000 non-null   object
18  Previous_Medication                  763 non-null    object
19  Recommended_Medication               1000 non-null   object
dtypes: float64(3), int64(3), object(14)
memory usage: 156.4+ KB
```

Statistical Summary

In [10]: `df.describe()`

Out[10]:

	Temperature	Age	BMI	Humidity	AQI	Heart_Rate
count	1000.00000	1000.000000	1000.00000	1000.000000	1000.000000	1000.00000
mean	38.09890	51.152000	26.44320	60.594400	248.493000	80.14400
std	1.13887	28.873354	4.90115	17.058475	143.705244	11.79196
min	36.00000	1.000000	18.00000	30.000000	0.000000	60.00000
25%	37.20000	26.000000	22.40000	46.600000	127.750000	70.00000
50%	38.20000	51.000000	26.50000	60.950000	247.500000	80.00000
75%	39.10000	76.000000	30.52500	74.725000	375.250000	90.00000
max	40.00000	100.000000	35.00000	90.000000	499.000000	100.00000

Categorical summary

In [11]:

df.describe(include="object")

Out[11]:

	Fever_Severity	Gender	Headache	Body_Ache	Fatigue	Chronic_Conditions	Allerg
count	1000	1000	1000	1000	1000	1000	1000
unique	3	2	2	2	2	2	2
top	High Fever	Female	Yes	Yes	No	Yes	Yes
freq	532	512	515	515	508	516	515

In [12]:

df["Gender"].unique()

Out[12]:

array(['Female', 'Male'], dtype=object)

Check Missing Values

In [13]:

df.isnull().sum()

```
Out[13]: Temperature      0
Fever_Severity           0
Age                      0
Gender                   0
BMI                      0
Headache                 0
Body_Ache                 0
Fatigue                  0
Chronic_Conditions       0
Allergies                 0
Smoking_History           0
Alcohol_Consumption      0
Humidity                  0
AQI                       0
Physical_Activity         0
Diet_Type                 0
Heart_Rate                0
Blood_Pressure            0
Previous_Medication      237
Recommended_Medication    0
dtype: int64
```

```
In [14]: df["Previous_Medication"] = df["Previous_Medication"].fillna(df["Previous_Medication"])
```

```
In [15]: df.isnull().sum()
```

```
Out[15]: Temperature      0
Fever_Severity           0
Age                      0
Gender                   0
BMI                      0
Headache                 0
Body_Ache                 0
Fatigue                  0
Chronic_Conditions       0
Allergies                 0
Smoking_History           0
Alcohol_Consumption      0
Humidity                  0
AQI                       0
Physical_Activity         0
Diet_Type                 0
Heart_Rate                0
Blood_Pressure            0
Previous_Medication      0
Recommended_Medication    0
dtype: int64
```

```
In [16]: df.duplicated().sum()
```

```
Out[16]: np.int64(0)
```

```
In [17]: df.dtypes
```

```
Out[17]: Temperature      float64
Fever_Severity           object
Age                      int64
Gender                   object
BMI                      float64
Headache                 object
Body_Ache                object
Fatigue                  object
Chronic_Conditions       object
Allergies                object
Smoking_History          object
Alcohol_Consumption      object
Humidity                 float64
AQI                      int64
Physical_Activity        object
Diet_Type                object
Heart_Rate               int64
Blood_Pressure           object
Previous_Medication      object
Recommended_Medication   object
dtype: object
```

Descriptive Statistics - Temperature

```
In [18]: print("Mean:", df["Temperature"].mean())
print("Median:", df["Temperature"].median())
print("Mode:", df["Temperature"].mode()[0])
print("Minimum:", df["Temperature"].min())
print("Maximum:", df["Temperature"].max())
```

```
Mean: 38.0989
Median: 38.2
Mode: 39.2
Minimum: 36.0
Maximum: 40.0
```

Descriptive Statistics - Age

```
In [21]: print("Mean:", df["Age"].mean())
print("Median:", df["Age"].median())
print("Mode:", df["Age"].mode()[0])
print("Minimum:", df["Age"].min())
print("Maximum:", df["Age"].max())
```

```
Mean: 51.152
Median: 51.0
Mode: 89
Minimum: 1
Maximum: 100
```

1. Age Group Analysis

```
In [22]: df["Age_Group"] = pd.cut(
    df["Age"],
    bins=[0, 18, 31, 46, 61, 101],
    labels=["Child", "Young Adult", "Adult", "Middle Aged", "Senior"],
    right=False
)
```

```
In [23]: df[["Age", "Age_Group"]].head()
```

```
Out[23]:
```

	Age	Age_Group
0	89	Senior
1	94	Senior
2	92	Senior
3	66	Senior
4	28	Young Adult

```
In [24]: df["Age_Group"].value_counts()
```

```
Out[24]: Age_Group
Senior      411
Child       167
Middle Aged 150
Adult       145
Young Adult 127
Name: count, dtype: int64
```

Temperature Analysis

```
In [25]: df.groupby("Age_Group")["Temperature"].mean()
```

C:\Users\vaish\AppData\Local\Temp\ipykernel_4252\613700910.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
df.groupby("Age_Group")["Temperature"].mean()
```

```
Out[25]: Age_Group
Child      38.152695
Young Adult 38.048031
Adult      38.144828
Middle Aged 38.124667
Senior     38.067153
Name: Temperature, dtype: float64
```

Temperature v/s Fever Severity

```
In [26]: df.groupby("Fever_Severity")["Temperature"].mean()
```

```
Out[26]: Fever_Severity
High Fever  39.015226
Mild Fever  37.670352
Normal      36.603717
Name: Temperature, dtype: float64
```

Diet Type Analysis

```
In [27]: df["Diet_Type"].value_counts()
```

```
Out[27]: Diet_Type
Non-Vegetarian    348
Vegan             332
Vegetarian        320
Name: count, dtype: int64
```

Diet Type v/s Fever Severity

```
In [28]: df.groupby("Diet_Type")["Fever_Severity"].value_counts()
```

```
Out[28]: Diet_Type    Fever_Severity
Non-Vegetarian    High Fever        171
                  Normal            104
                  Mild Fever         73
Vegan             High Fever        182
                  Normal             85
                  Mild Fever         65
Vegetarian        High Fever        179
                  Normal             80
                  Mild Fever         61
Name: count, dtype: int64
```

Age Group v/s Fever Severity

```
In [29]: df.groupby("Age_Group")["Fever_Severity"].value_counts()
```

C:\Users\vaish\AppData\Local\Temp\ipykernel_4252\267068831.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
df.groupby("Age_Group")["Fever_Severity"].value_counts()
```

```
Out[29]: Age_Group    Fever_Severity
Child              High Fever        95
                  Normal            46
                  Mild Fever        26
Young Adult       High Fever        65
                  Normal            32
                  Mild Fever        30
Adult              High Fever        81
                  Normal            40
                  Mild Fever        24
Middle Aged       High Fever        82
                  Normal            43
                  Mild Fever        25
Senior             High Fever       209
                  Normal           108
                  Mild Fever        94
Name: count, dtype: int64
```

```
In [45]: df.groupby("Allergies")["Fever_Severity"].value_counts()
```

```
Out[45]: Allergies    Fever_Severity
No              High Fever      276
              Normal          145
              Mild Fever       101
Yes            High Fever      256
              Normal          124
              Mild Fever        98
Name: count, dtype: int64
```

Recommended Medicine Dataset

```
In [30]: df["Recommended_Medication"].value_counts()
```

```
Out[30]: Recommended_Medication
Ibuprofen      801
Paracetamol    199
Name: count, dtype: int64
```

Multivariate Analysis

```
In [31]: df.groupby(["Age_Group", "Fever_Severity"])["Temperature"].mean()
```

C:\Users\vaish\AppData\Local\Temp\ipykernel_4252\499295332.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
df.groupby(["Age_Group", "Fever_Severity"])["Temperature"].mean()
```

```
Out[31]: Age_Group    Fever_Severity
Child      High Fever      39.015789
           Mild Fever      37.703846
           Normal          36.623913
Young Adult High Fever      38.993846
           Mild Fever      37.610000
           Normal          36.537500
Adult      High Fever      39.033333
           Mild Fever      37.691667
           Normal          36.617500
Middle Aged High Fever      39.058537
           Mild Fever      37.692000
           Normal          36.595349
Senior     High Fever      38.997608
           Mild Fever      37.669149
           Normal          36.612963
Name: Temperature, dtype: float64
```

```
In [32]: df.groupby(["Age_Group", "Fever_Severity"])["Heart_Rate"].mean()
```

C:\Users\vaish\AppData\Local\Temp\ipykernel_4252\465305356.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
df.groupby(["Age_Group", "Fever_Severity"])["Heart_Rate"].mean()
```

```
Out[32]: Age_Group    Fever_Severity
Child      High Fever    79.252632
           Mild Fever    78.115385
           Normal        80.391304
Young Adult High Fever    82.923077
           Mild Fever    76.466667
           Normal        80.593750
Adult      High Fever    81.037037
           Mild Fever    80.708333
           Normal        78.650000
Middle Aged High Fever    79.036585
           Mild Fever    77.040000
           Normal        79.209302
Senior     High Fever    79.818182
           Mild Fever    82.287234
           Normal        80.981481
Name: Heart_Rate, dtype: float64
```

Overall Recommended Medicine

```
In [33]: df.groupby("Recommended_Medication")["Fever_Severity"].value_counts()
```

```
Out[33]: Recommended_Medication    Fever_Severity
Ibuprofen                        High Fever      532
                                Normal          269
Paracetamol                      Mild Fever      199
Name: count, dtype: int64
```

Diet Type by Age group

```
In [36]: df.groupby("Age_Group")["Diet_Type"].value_counts()
```

C:\Users\vaish\AppData\Local\Temp\ipykernel_4252\55551483.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
df.groupby("Age_Group")["Diet_Type"].value_counts()
```

```
Out[36]: Age_Group    Diet_Type
Child      Non-Vegetarian    61
           Vegan            53
           Vegetarian        53
Young Adult Vegetarian        53
           Non-Vegetarian    42
           Vegan            32
Adult      Non-Vegetarian    50
           Vegetarian        48
           Vegan            47
Middle Aged Vegan            58
           Non-Vegetarian    46
           Vegetarian        46
Senior     Non-Vegetarian    149
           Vegan            142
           Vegetarian        120
Name: count, dtype: int64
```

Average Age by Recommended Medicine

```
In [37]: df.groupby("Recommended_Medication")["Age"].mean()
```

```
Out[37]: Recommended_Medication
Ibuprofen      50.324594
Paracetamol    54.482412
Name: Age, dtype: float64
```

Average Temperature by Recommended Medicine

```
In [38]: df.groupby("Recommended_Medication")["Temperature"].mean()
```

```
Out[38]: Recommended_Medication
Ibuprofen      38.205368
Paracetamol    37.670352
Name: Temperature, dtype: float64
```

Average Heart rate by Fever Severity

```
In [39]: df.groupby("Fever_Severity")["Heart_Rate"].mean()
```

```
Out[39]: Fever_Severity
High Fever    80.161654
Mild Fever    80.015075
Normal        80.204461
Name: Heart_Rate, dtype: float64
```

FILTERING

Find the patients whose temperature is greater than 39 degree celsius

```
In [40]: high_temperature= df[df["Temperature"]>39]
high_temperature.head()
```

```
Out[40]:
```

	Temperature	Fever_Severity	Age	Gender	BMI	Headache	Body_Ache	Fatigue	Chro
3	39.8	High Fever	66	Male	18.7	No	Yes	No	
4	39.3	High Fever	28	Male	21.0	No	Yes	No	
6	39.1	High Fever	63	Male	18.5	Yes	No	No	
7	39.9	High Fever	54	Female	20.7	No	Yes	Yes	
10	39.5	High Fever	24	Male	33.0	Yes	Yes	No	

5 rows × 21 columns



Find the patients who are 50 years older

```
In [41]: older_patients =df[df["Age"]>=50]
older_patients.head(10)
```

Out[41]:

	Temperature	Fever_Severity	Age	Gender	BMI	Headache	Body_Ache	Fatigue	Chro
0	36.1	Normal	89	Female	24.0	No	Yes	No	
1	37.5	Mild Fever	94	Male	26.6	No	No	No	
2	36.4	Normal	92	Male	27.8	No	No	No	
3	39.8	High Fever	66	Male	18.7	No	Yes	No	
6	39.1	High Fever	63	Male	18.5	Yes	No	No	
7	39.9	High Fever	54	Female	20.7	No	Yes	Yes	
11	36.5	Normal	75	Male	23.1	Yes	Yes	Yes	
13	38.1	High Fever	97	Male	20.9	No	Yes	No	
14	39.7	High Fever	83	Female	27.0	No	Yes	Yes	
17	38.5	High Fever	79	Female	34.7	Yes	Yes	No	

10 rows × 21 columns



PIVOT TABLE

```
In [42]: pivot= pd.pivot_table(df,
                                index="Age_Group",
                                columns="Fever_Severity",
                                aggfunc="size"
                                )

pivot
```

C:\Users\vaish\AppData\Local\Temp\ipykernel_4252\3040418662.py:1: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior

```
    pivot= pd.pivot_table(df,
```

Out[42]: **Fever_Severity** **High Fever** **Mild Fever** **Normal**

Age_Group			
Child	95	26	46
Young Adult	65	30	32
Adult	81	24	40
Middle Aged	82	25	43
Senior	209	94	108

correlation Analysis

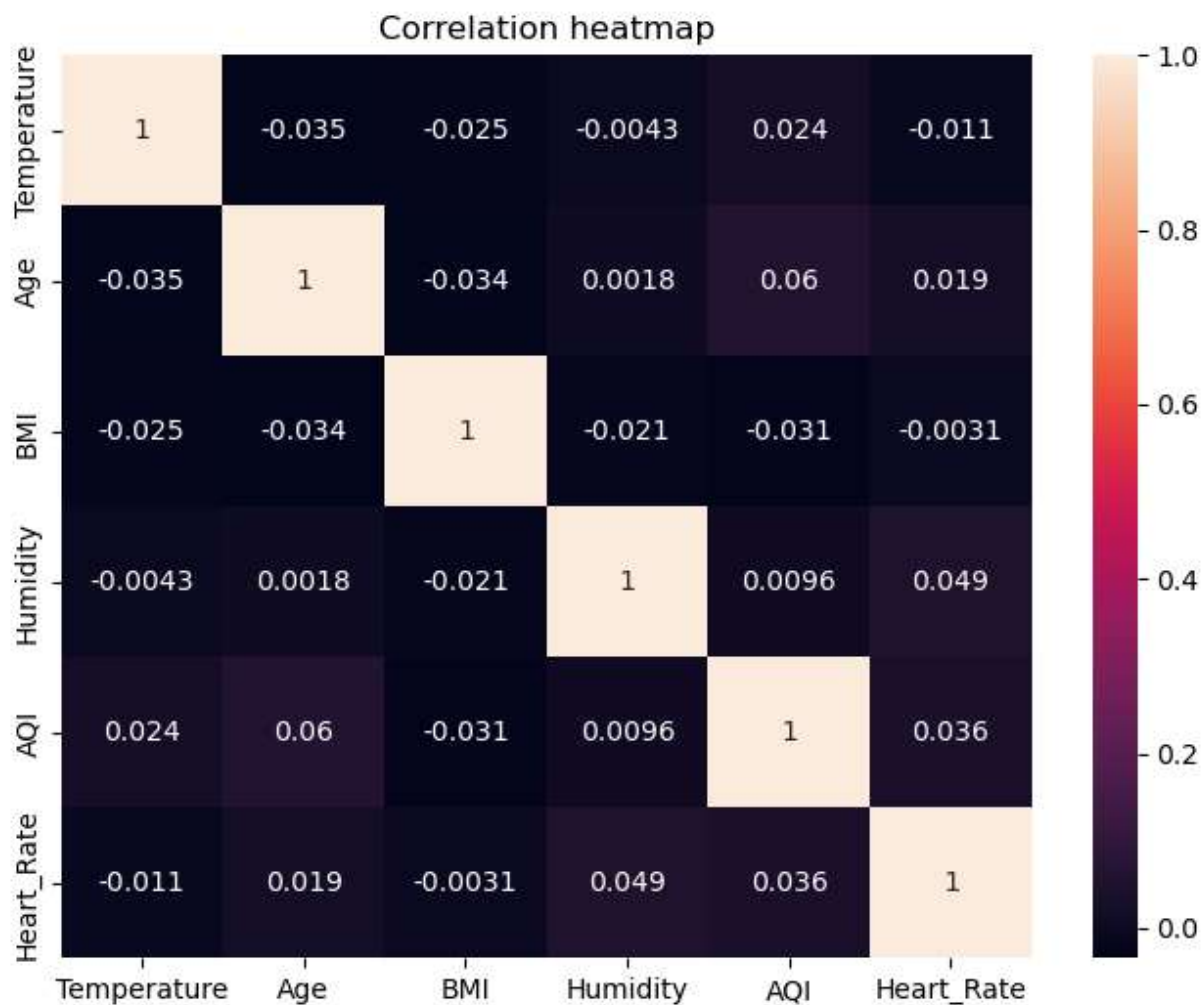
```
In [43]: correlation=df.corr(numeric_only=True)
correlation
```

Out[43]:

	Temperature	Age	BMI	Humidity	AQI	Heart_Rate
Temperature	1.000000	-0.034607	-0.024888	-0.004297	0.023651	-0.011318
Age	-0.034607	1.000000	-0.034103	0.001818	0.059965	0.018755
BMI	-0.024888	-0.034103	1.000000	-0.020887	-0.030511	-0.003137
Humidity	-0.004297	0.001818	-0.020887	1.000000	0.009569	0.048946
AQI	0.023651	0.059965	-0.030511	0.009569	1.000000	0.035997
Heart_Rate	-0.011318	0.018755	-0.003137	0.048946	0.035997	1.000000

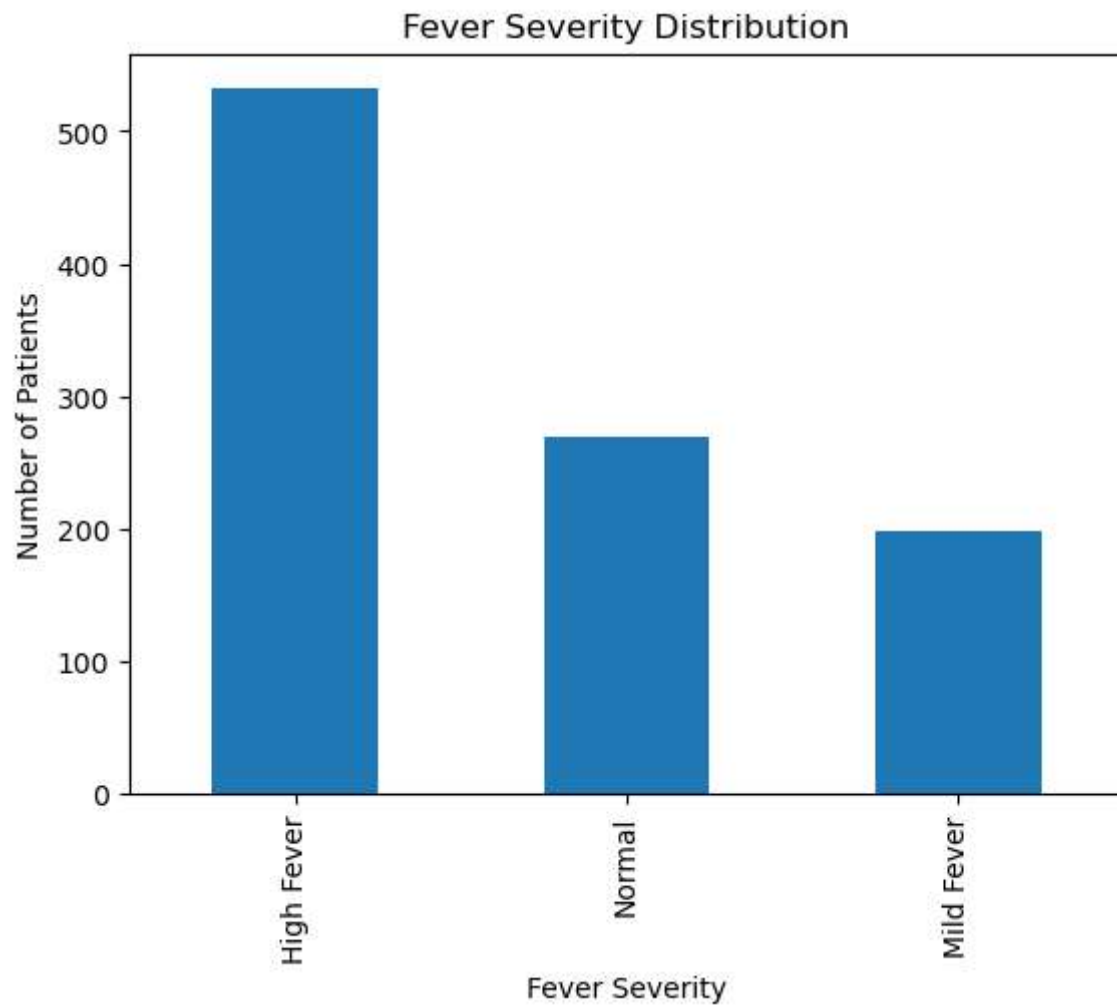
Visualizations|

```
In [44]: plt.figure(figsize=(8,6))
sns.heatmap(correlation,annot=True)
plt.title("Correlation heatmap")
plt.show()
```



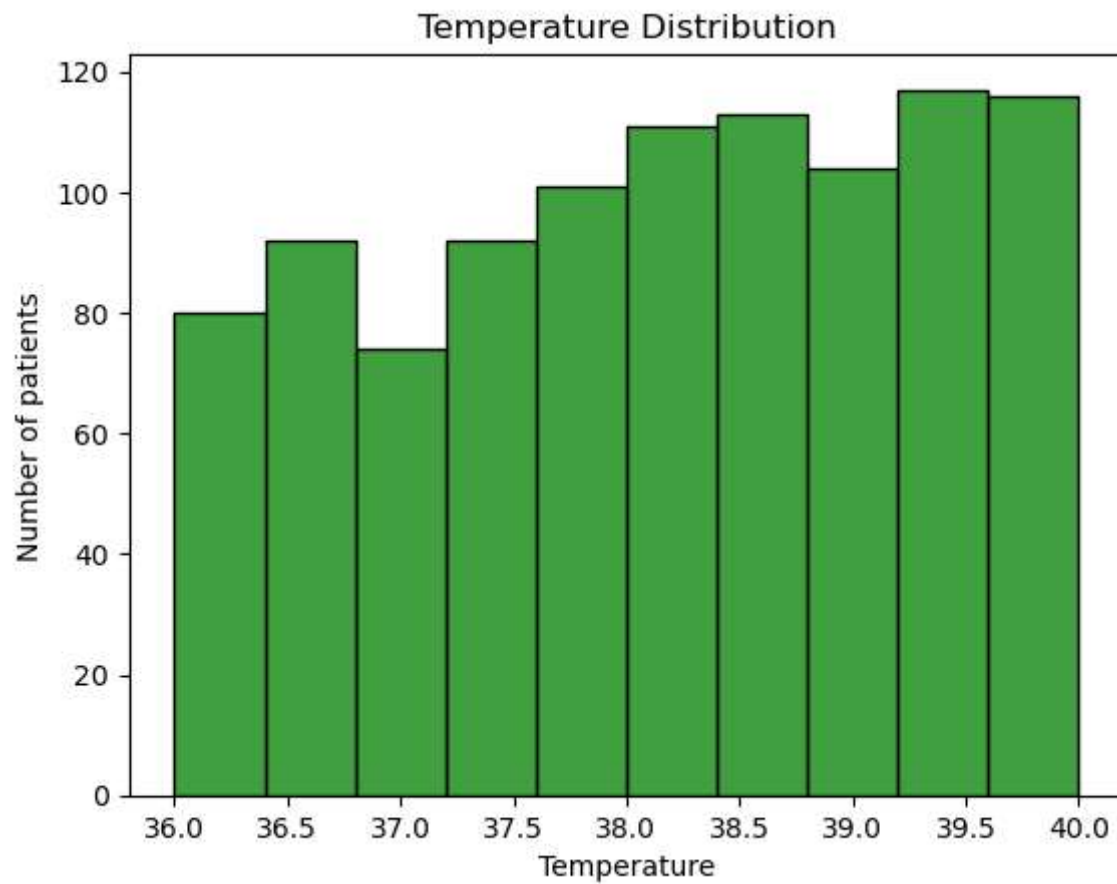
####finding The correlation analysis shows that the numerical variables have mostly weak relationships with each other. The highest positive correlation is between Age and AQI (0.060), while Humidity and Heart Rate also show a very weak positive correlation (0.049). Overall, no strong linear relationship is observed among the numerical variables.

```
In [39]: df["Fever_Severity"].value_counts().plot.bar()
plt.title("Fever Severity Distribution")
plt.xlabel("Fever Severity")
plt.ylabel("Number of Patients")
plt.show()
```



finding High fever has the highest no of patients ,while mild fever has the lowest number of patients.

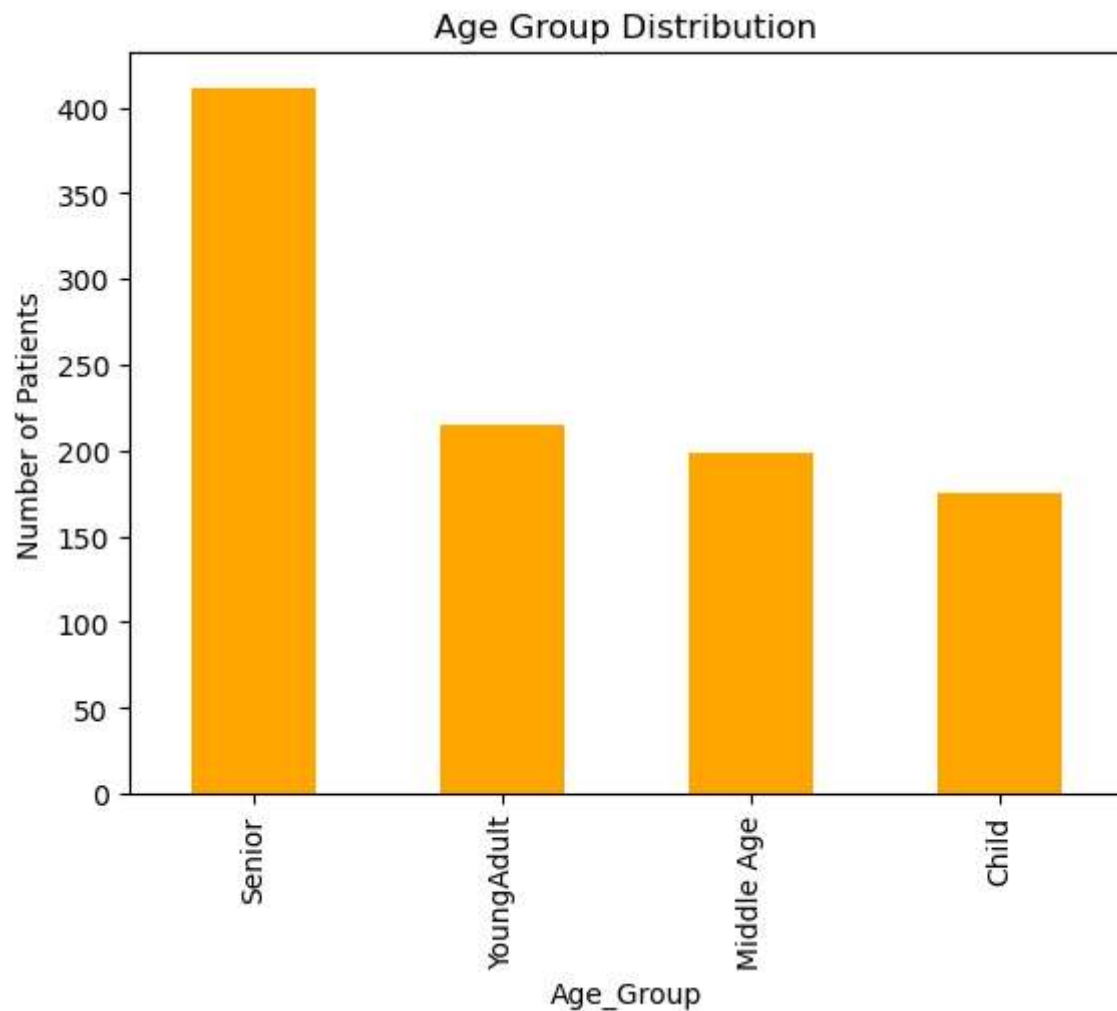
```
In [40]: sns.histplot(df["Temperature"], bins=10 , color="green")
plt.xlabel("Temperature")
plt.ylabel("Number of patients")
plt.title("Temperature Distribution")
plt.show()
```



The histogram shows distribution of patients across different temperature ranges. Most patients have temperatures between approximately 38°C and 40°C.

```
In [41]: df["Age_Group"].value_counts().plot.bar(color="orange")

plt.xlabel("Age_Group")
plt.ylabel("Number of Patients")
plt.title("Age Group Distribution")
plt.show()
```

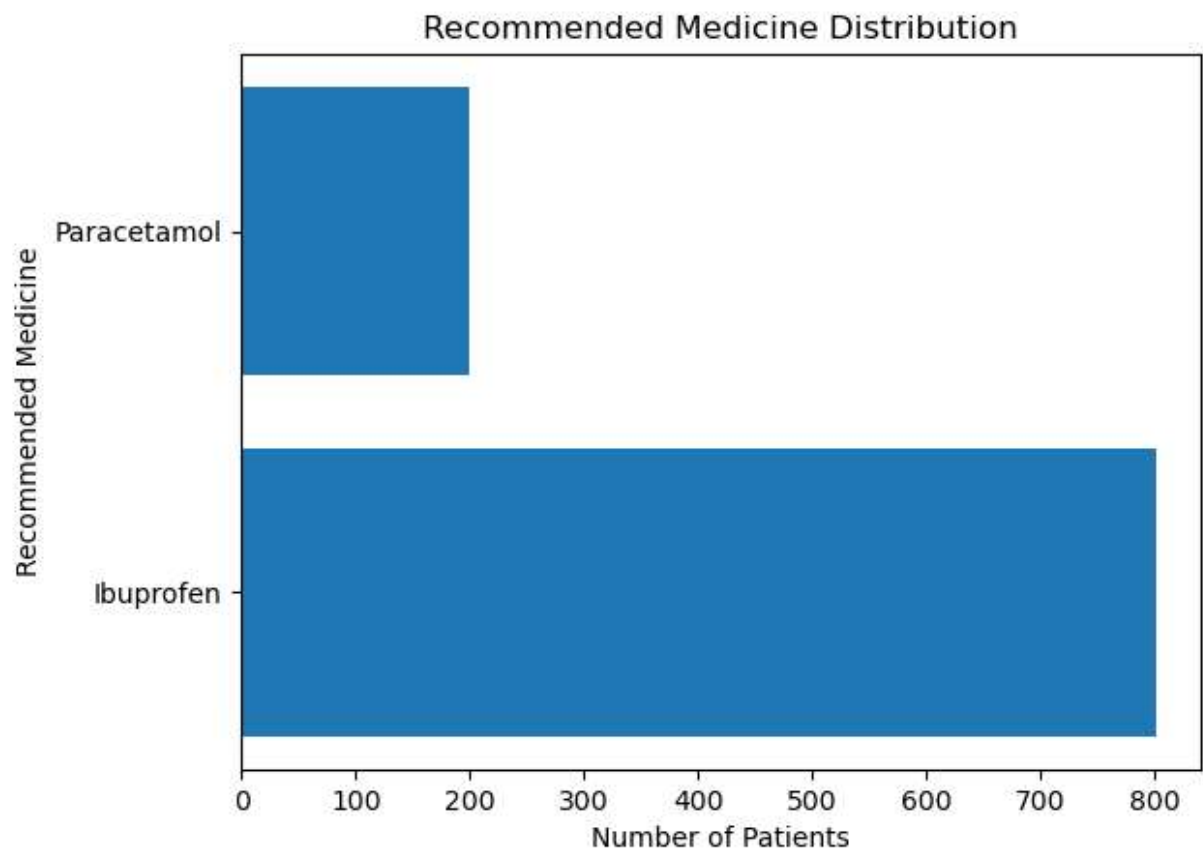


Senior patients had the highest number of records, while child patients had the lowest number of records among the four age groups.

```
In [54]: medicine_counts = df["Recommended_Medication"].value_counts()
plt.barh(medicine_counts.index, medicine_counts.values)

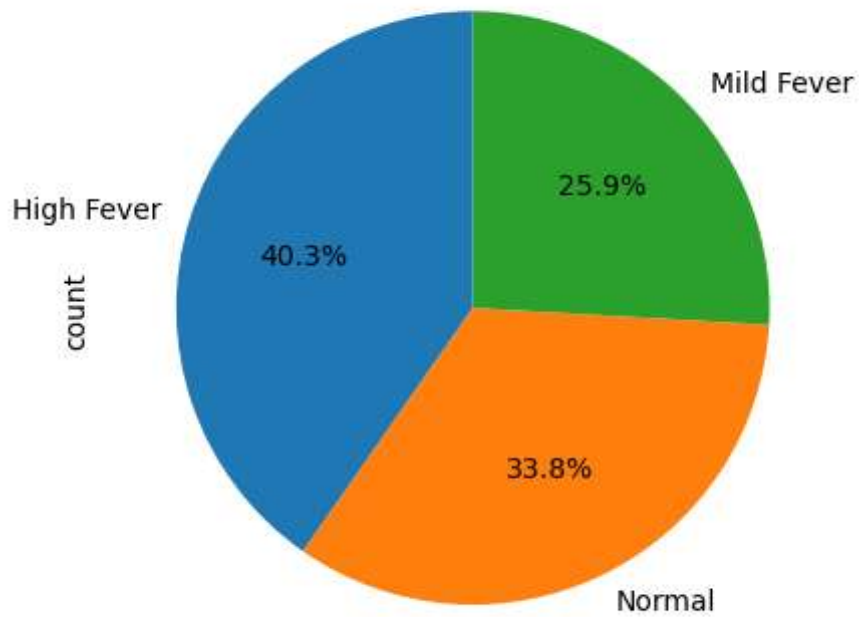
plt.title("Recommended Medicine Distribution")
plt.xlabel("Number of Patients")
plt.ylabel("Recommended Medicine")

plt.show()
```



```
In [42]: df["Temperature_Category"].value_counts().plot.pie(  
autopct="%1.1f%%", startangle=90)  
colors=sns.color_palette("bright")  
plt.title("Temperature Category Distribution")  
plt.show()
```

Temperature Category Distribution

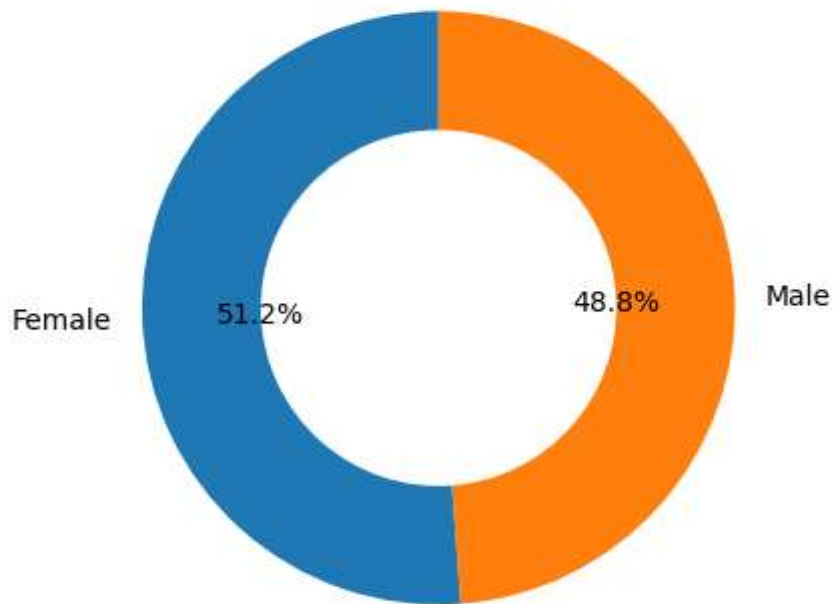


High fever was the common temperature category, followed by Normal and Mild fever.

```
In [43]: df["Gender"].value_counts().plot.pie(
        autopct="%1.1f%%",
        startangle=90,
        wedgeprops=dict(width=0.4)
    )

plt.ylabel("")
plt.title("Gender Distribution")
plt.show()
```

Gender Distribution



The gender distribution was nearly balanced, with female patients slightly higher than male patients.

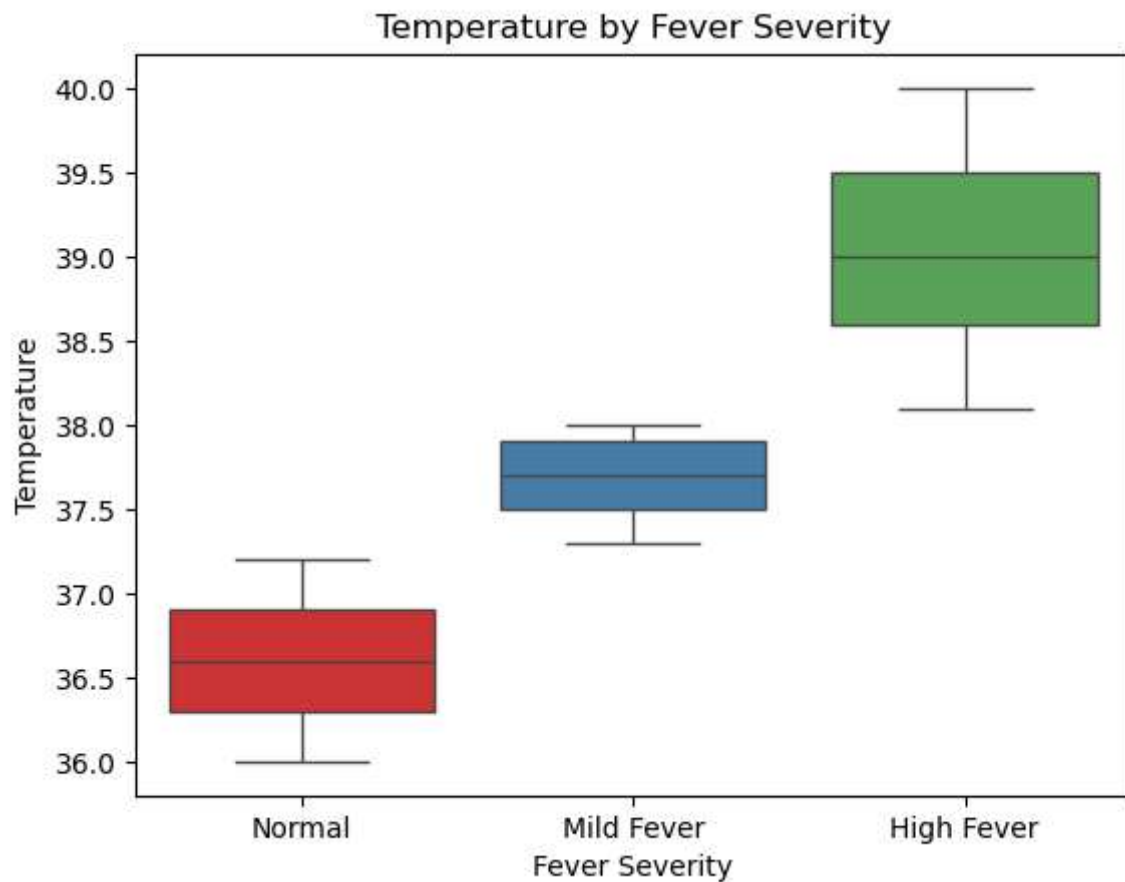
```
In [44]: sns.boxplot(x="Fever_Severity",y="Temperature",data=df, palette="Set1")

plt.xlabel("Fever Severity")
plt.ylabel("Temperature")
plt.title("Temperature by Fever Severity")
plt.show()
```

C:\Users\vaish\AppData\Local\Temp\ipykernel_3460\4116514370.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

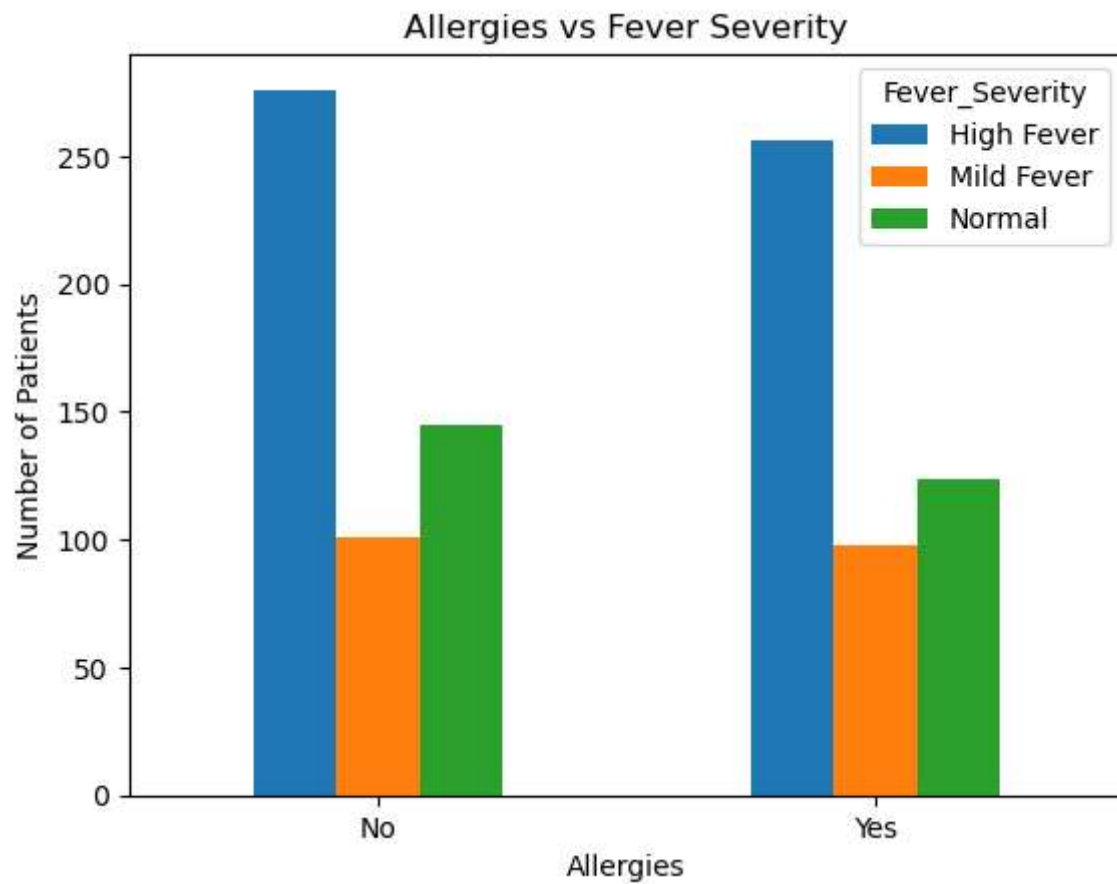
```
sns.boxplot(x="Fever_Severity",y="Temperature",data=df, palette="Set1")
```



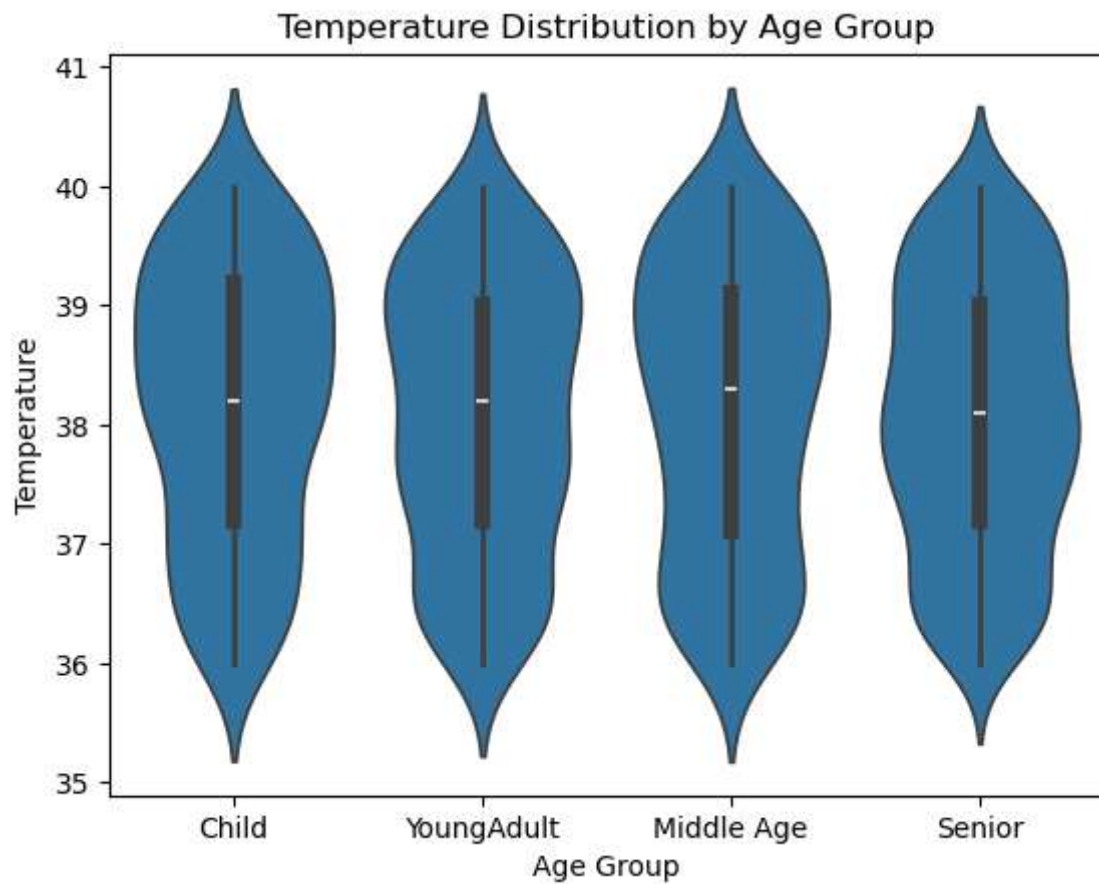
Temperature increased from Normal to Mild Fever and was highest for High fever.

```
In [47]: allergy_severity = pd.crosstab(df["Allergies"], df["Fever_Severity"])
allergy_severity.plot(kind="bar")
plt.title("Allergies vs Fever Severity")
plt.xlabel("Allergies")
plt.ylabel("Number of Patients")
plt.xticks(rotation=0)

plt.show()
```

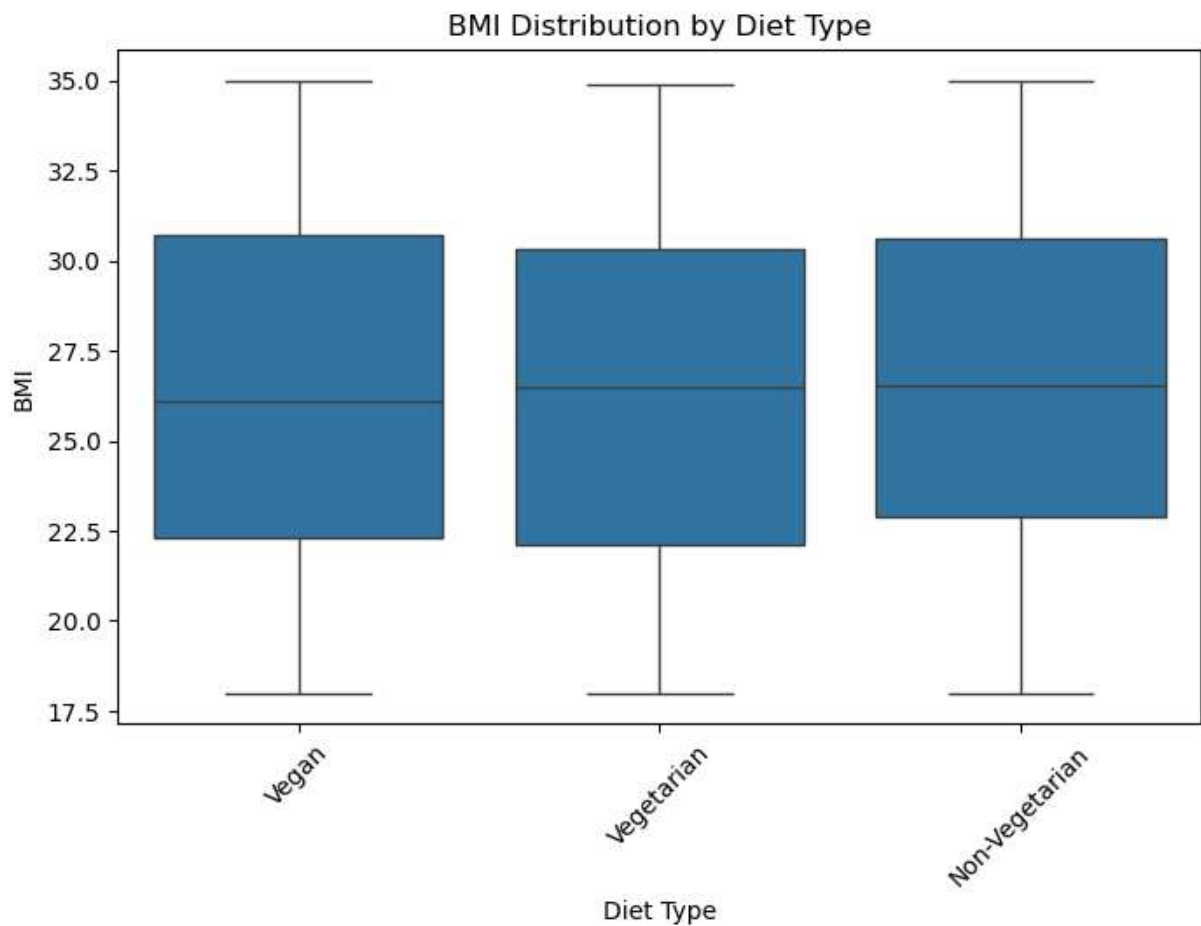


```
In [46]: sns.violinplot(x="Age_Group", y="Temperature", data=df)
plt.xlabel("Age Group")
plt.ylabel("Temperature")
plt.title("Temperature Distribution by Age Group")
plt.show()
```



Temperature Distribution were similar across all age groups, with values spread across a similar range.

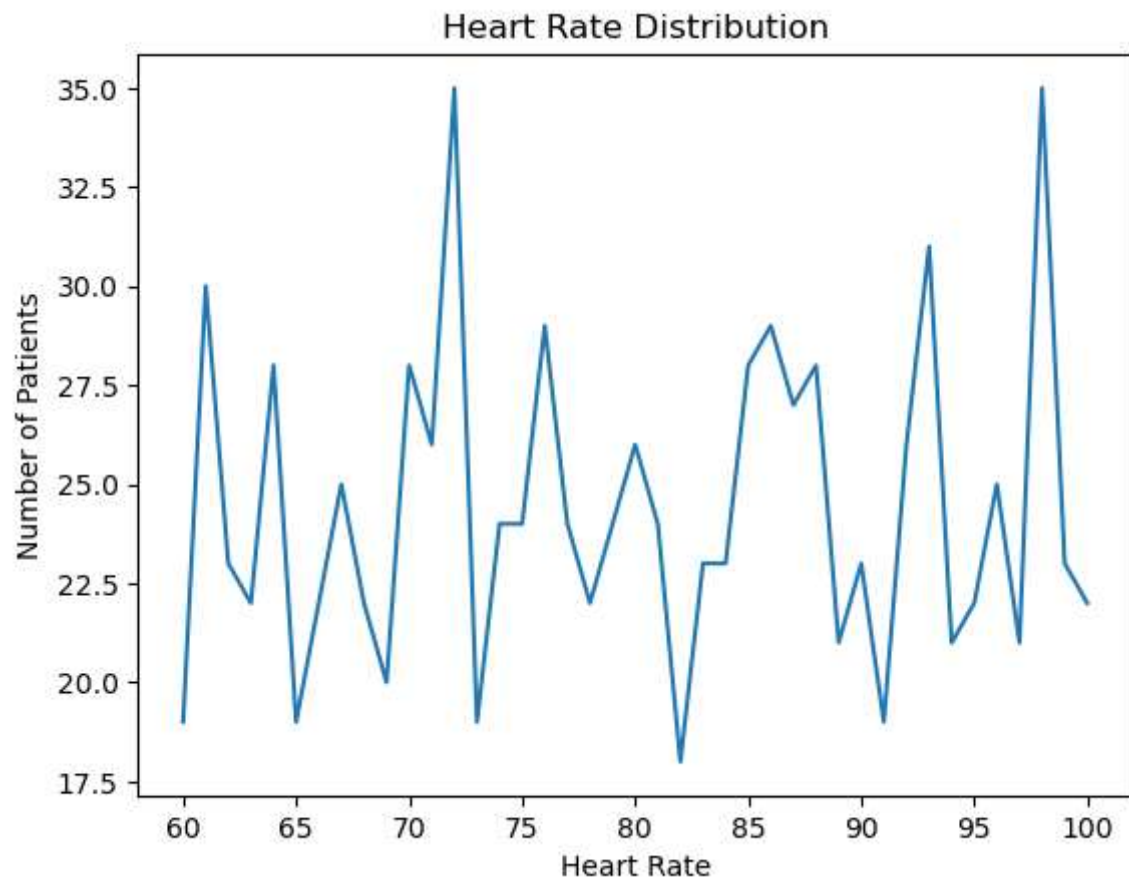
```
In [56]: plt.figure(figsize=(8,5))
sns.boxplot(data=df, x="Diet_Type", y="BMI")
plt.title("BMI Distribution by Diet Type")
plt.xlabel("Diet Type")
plt.ylabel("BMI")
plt.xticks(rotation=45)
plt.show()
```



BMI distribution is relatively similar across the three diet types, with the Vegan group showing a slightly higher median BMI.

```
In [48]: heart_rate_counts= df["Heart_Rate"].value_counts().sort_index()

plt.plot(heart_rate_counts.index,heart_rate_counts.values)
plt.xlabel("Heart Rate")
plt.ylabel("Number of Patients")
plt.title("Heart Rate Distribution")
plt.show()
```

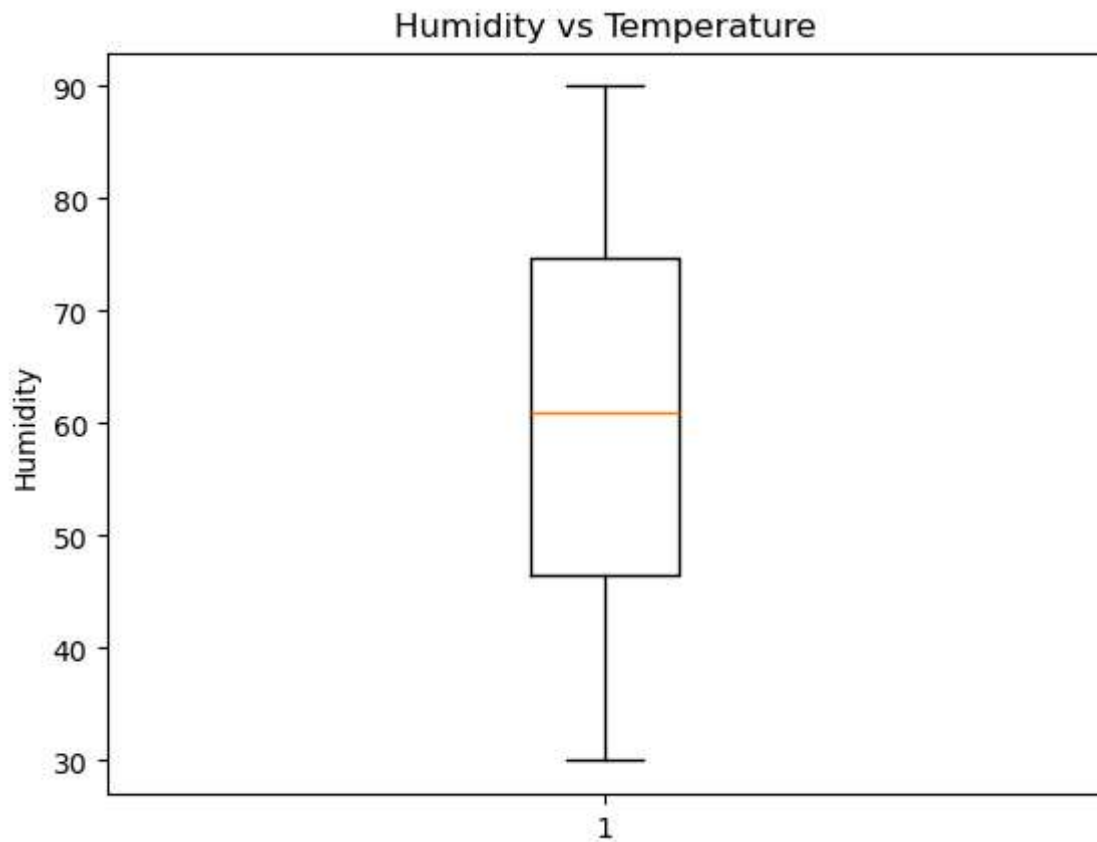


Heart rate values ranged from approximately 60 to 100 , with patient counts varying across the range.

```
In [49]: plt.boxplot(df["Humidity"])

plt.ylabel("Humidity")
plt.title("Humidity vs Temperature")

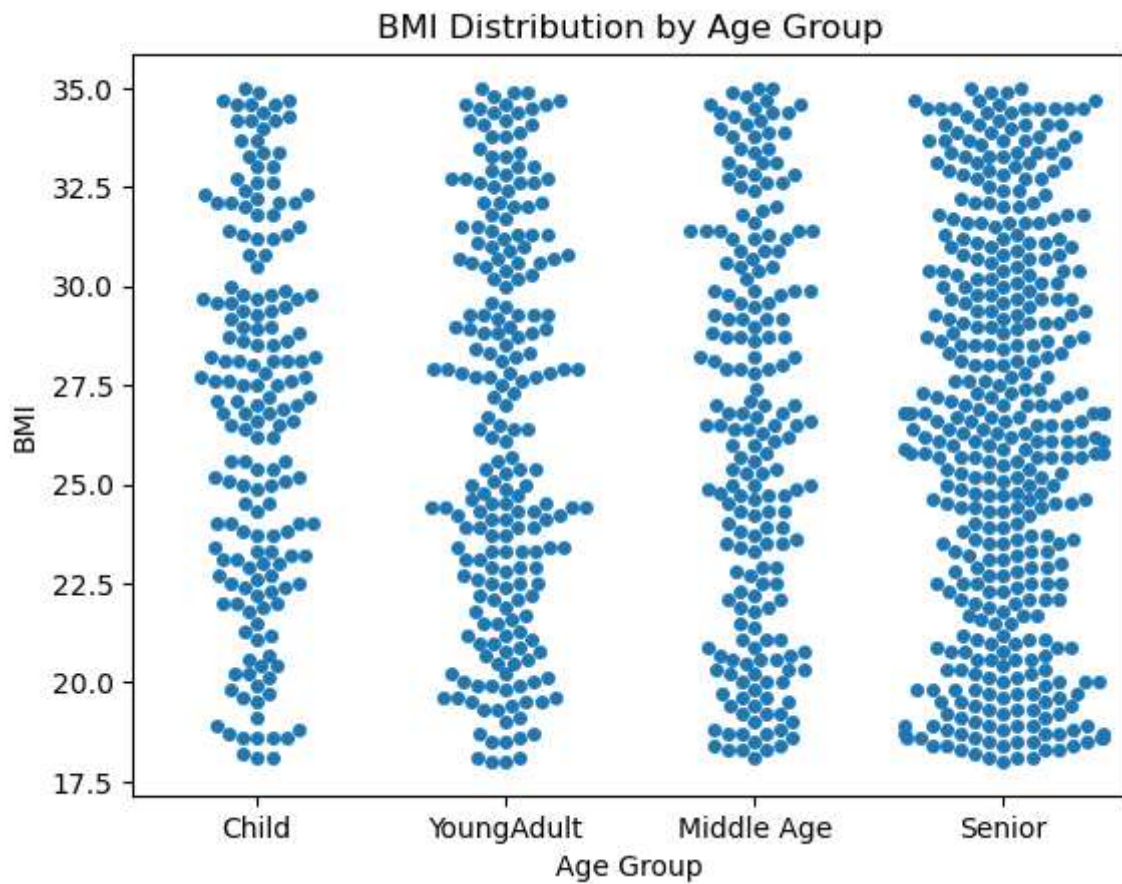
plt.show()
```



Humidity values range from approximately 29 to 90, with most values concentrated between 45 and 75.

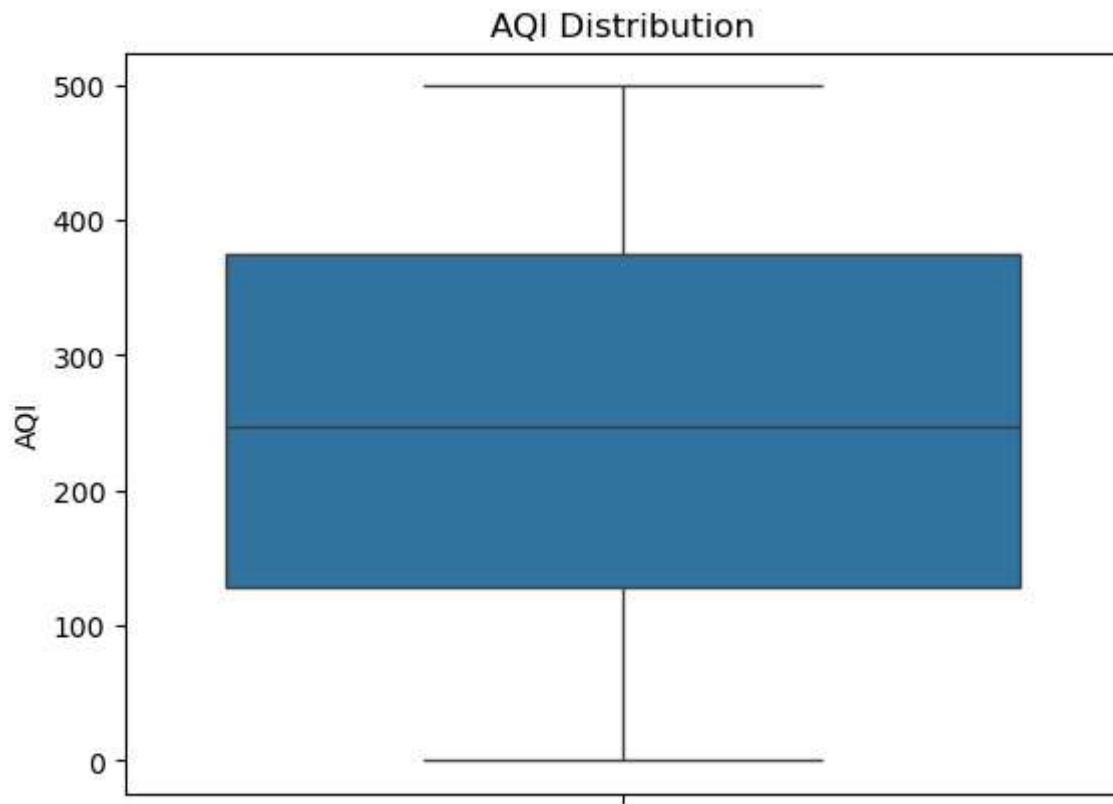
```
In [50]: sns.swarmplot(x="Age_Group", y="BMI", data=df)

plt.xlabel("Age Group")
plt.ylabel("BMI")
plt.title("BMI Distribution by Age Group")
plt.show()
```



BMI values vary across all age groups. Senior patients group shows a wider distribution of BMI values compared to the other age groups.

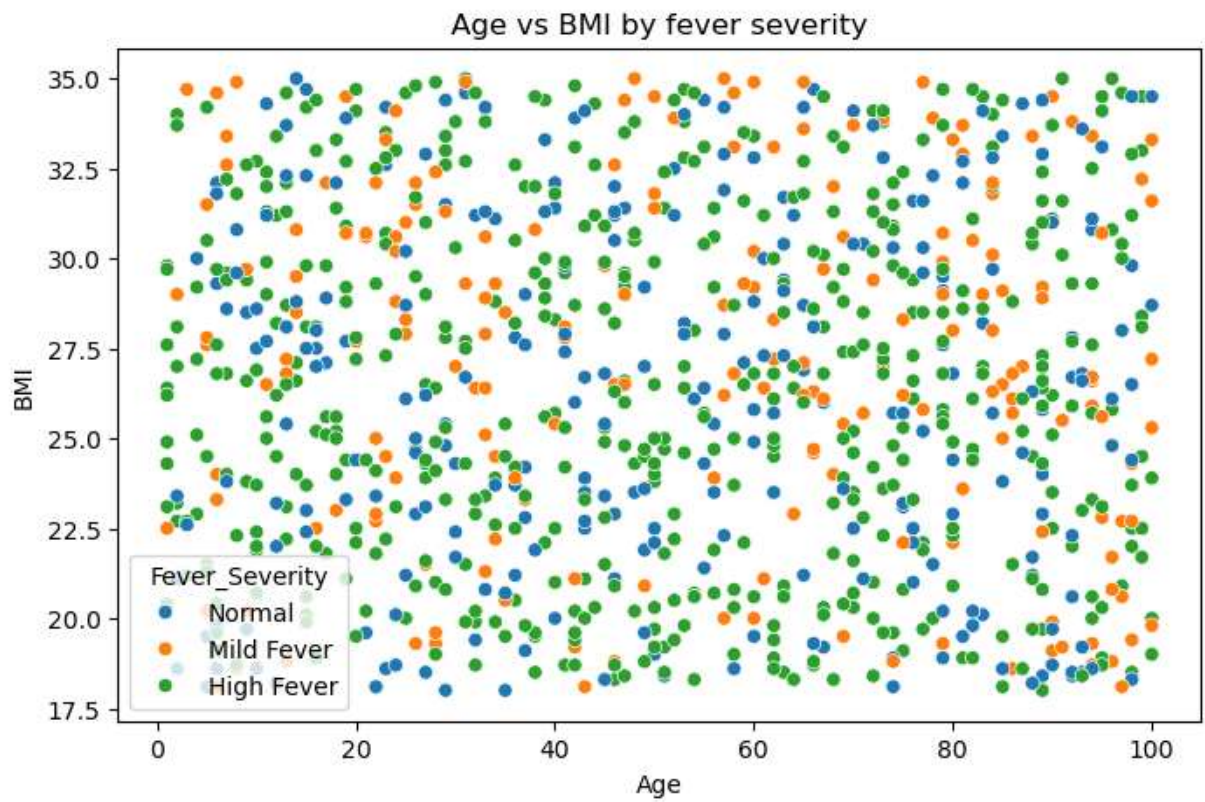
```
In [51]: sns.boxplot(y=df["AQI"])
plt.ylabel("AQI")
plt.title("AQI Distribution")
plt.show()
```



AQI values show a wide range with a median of around 250.

```
In [63]: plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x="Age", y="BMI", hue="Fever_Severity")
plt.title("Age vs BMI by fever severity")
plt.xlabel("Age")
plt.ylabel("BMI")

plt.show()
```



Age and BMI show no clear relationship, and fever severity is spread across different age and BMI values.”